표 1 선행 연구의 설계 비교

Table 1 Design properties of prior work

Work	Time	Space	Region cap	Hard deadline	Re-solve
Wiesner, CarbonScaler, GAIA [16–18]	○	×	×	△	×
CICS [3]	○	×	△	×	×
CASPER [4]	×	○	△	—	—
Sukprasert [6]	○	○	△	○	×
CarbonClipper [21]	○	○	×	○	—
Flex [7]	○	×	○	△	○
Caspian [8]	○	○	○	△	○
Zanotto [9]	○	○	○	○	×
CAST	○	○	△	○	○

2. 관련 연구

탄소 인지 스케줄링은 시간 축에서 먼저 연구되었다. Wiesner 등은 한 데이터센터 안에서 작업을 미루는 것만으로 얻는 절감을 보였고[16], CarbonScaler는 한 리전 안에서 작업의 서버 수를 탄소 예보에 맞춰 늘리고 줄이며[17], GAIA는 한 리전 클러스터에서 탄소, 완료 시간, 비용 사이의 맞바꿈을 다룬다[18]. 이 연구들은 작업이 실행되는 리전을 바꾸지 않는다. 지리적 부하 분산 연구는 그보다 먼저 요청과 배치 작업을 데이터센터 사이로 나눠 보내 전기요금이나 화석 에너지 사용을 줄이는 문제를 다뤘다[23–26]. 리전과 시각을 함께 옮기면서 리전 용량과 마감을 지키려 하면, 선행 연구에는 다음 세 가지가 걸린다.

첫째, 리전과 시각을 함께 옮기는 연구는 리전 용량과 마감 가운데 하나를 제약으로 두지 않는다. Sukprasert 등은 용량이 공간 이동의 이득을 좌우함을 확인하고도 두 축을 결합한 분석에서는 용량을 두지 않았고[6], Attenni 등도 자원이 항상 가용하다고 가정한다[20]. CarbonClipper는 작업 하나를 마감 안에서 리전 사이로 옮기는 온라인 문제를 다루며, 여러 작업이 자원을 나눠 쓰는 상황과 데이터센터 용량은 정식화에 없다[21]. Caspian은 반대로 용량을 하드 제약으로 두고 마감을 목적함수의 페널티로 완화한다[8].

둘째, 용량을 두는 연구는 대개 작업이 들어오는 순간에 한 번 배정을 정하고 다시 보지 않는다. Zanotto 등은 리전별 동시 실행 상한과 작업별 하드 마감을 함께 두고, 요청이 도착할 때마다 이미 정한 배정을 모두 보며 그 요청의 리전과 시각을 정수 계획으로 정한 뒤 다시 풀지 않는다[9]. CICS는 하루 전에 산정한 자원 곡선으로 다음 날의 유연한 부하를 묶어 개별 작업의 마감을 보지 않으며[3], Flex는 단일 클러스터 안에서 용량을 보지 않는 시간 이동이 저탄소 시각에 부하를 모은다는 점을 지적했으나 리전 사이의 이동은 다루지 않는다[7]. 매 기간 앞날 전체를 다시 최적화하고 당장의 결정만 실행하는 롤링 호라이즌 방식은 생산 계획과 모형 예측 제어에서 쓰여 온 표준 구조이며[13–15], 확정 시점으로 나누면 Zanotto 등의 방식은 도착 시 확정, CAST는 시작 시 확정에 해당한다[27].

셋째, 탄소와 그 밖의 목적 사이의 가중치를 사람이 고정한다. CASPER는 탄소와 서버 프로비저닝 비용 사이의 가중치를 고정하고 다른 값에서는 결과가 달라진다고 밝혔다[4]. CICS도 탄소 비용과 피크 전력 비용의 가중합을 쓰며[3], 두 축을 하나의 정수 계획으로 푸는 Caspian도 가중치를 사람이 정한다[8]. 다목적 최적화에서는 목적값들이 모두 가장 작은 이상점에 가장 가까운 해를 고르는 방법이 쓰인다[30].

표 1은 선행 연구의 이런 성질을 모은 것이다. ○는 있음, △는 페널티나 부분 분석, ×는 없음, —는 해당 없음을 뜻한다.